What must be remembered about a deleted character?

The minimal counterexample, with branches, merges, and the LCA
(working note — throwaway)

Sal / FP Launchpad — KC + Claude

2026-07-14

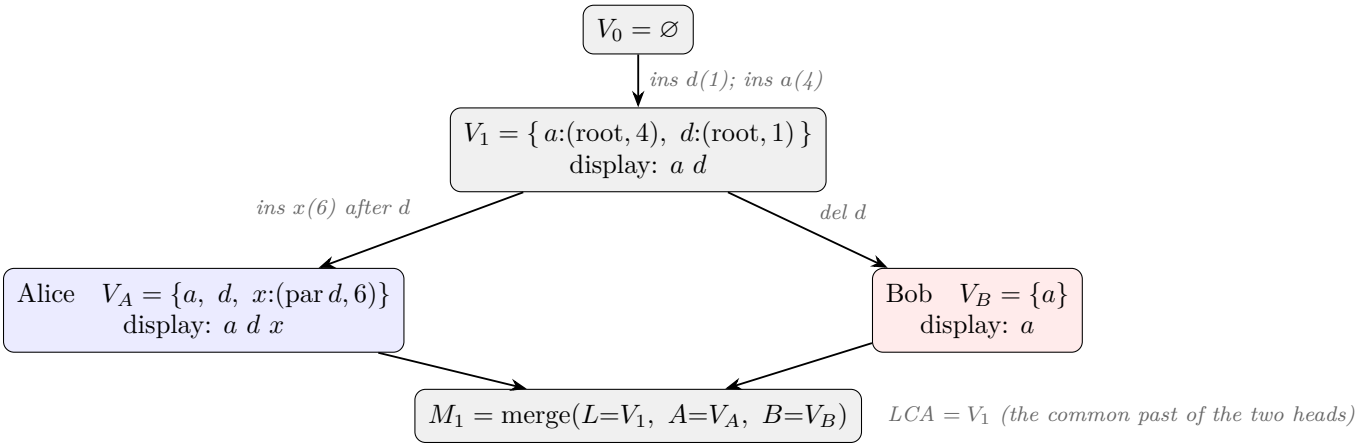
The question. When a character is deleted from a replicated list, what must the system remember about it, and for how long? The requirement throughout: once any user has seen character u displayed before character v, no user may ever see v before u; and replicas with the same events must agree.

The cast. Three characters, identified by *when* they were typed (larger = later): d at time 1, a at time 4, x at time 6. Concurrent same-position characters display newest-first, so a document containing a and d at the same position shows "a d".

1 The four-event history

step	action	screen	
1	type d (time 1)	d	
2	type a (time 4), same position	a d	
	— *both replicas sync* —	a d	
3	**Alice**: type x (time 6) *after* d	a d x	⇒ **Alice has seen a before x**
4	**Bob**: delete d	a	
5	*merge*	?	

2 The version DAG and the first merge



What M_1 sees, record by record:

node	in L $(= V_1)$	in A	in B	verdict
a (4)	root	root	root	in all three ⇒ **lives**
d (1)	root	root	**absent**	live in LCA, missing in B ⇒ B deleted it ⇒ **dies**
x (6)	absent	par $= d$	absent	not in LCA, born in A ⇒ **lives**

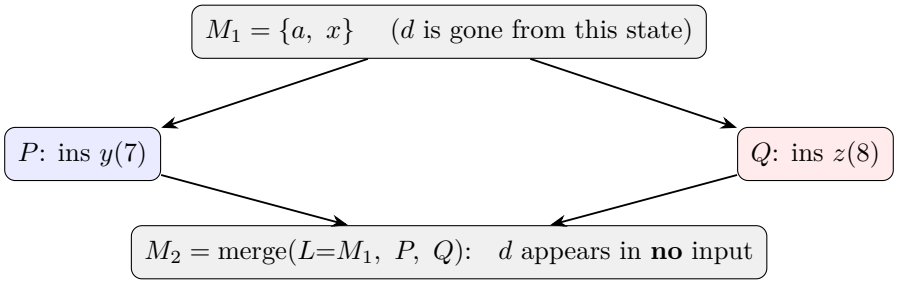
Survivors $\{a, x\}$. The whole question is the root order: a **before** x**, or** x **before** a**?**

design	root comparison	result	
RGA (keep d as tombstone)	$a(4)$ vs $d(1)$: a first; x rides after d; read skips d	$a\ x$	✓
always keep the dead time	x's rank $= (1, 6)$ via dead parent; 4 vs $(1, 6)$: $4 > 1$	$a\ x$	✓
keep only if d conflicted	d never conflicted ⇒ nothing kept; 4 vs (6): $6 > 4$	$x\ a$	×

The conditional design reverses the order Alice already saw. The point: *the position of x relative to a is decided by the timestamp of d* — a character that is gone, and that looked completely uninteresting at deletion time (typed sequentially, never part of any conflict). Any rule that skips remembering in "boring" cases skips exactly this one. (A second, six-event counterexample kills the smarter condition "remember d if something displays above it": that condition can change *after* d's death, via a concurrent insert in a replica where d still exists, so replicas answer it differently.)

3 Where the LCA helps, and where it stops helping

At M_1, d's timestamp is still present in two of the three inputs — the LCA remembers it. So the *first* merge after a death could answer correctly with no retention at all, just by consulting its inputs; the conditional design fails at M_1 by rule, not for lack of information. Retention is forced *one merge later*:



M_2 must still order a vs x, and no input contains d. Whatever M_1 learned had to be written into its *output state*. That is exactly what the working designs do. After M_1:

RGA: $d^\dagger$ kept in the sequence ours: $x : (\text{par} = \text{root},\ \text{rank} = (1, 6))$

and M_2 reads $z\ y\ a\ x$ in both (machine-checked).

Division of labor.

- **The LCA covers the first merge after a death** — "dead in a branch, live in the LCA" is visible for free, timestamp included. This is real power that two-way-merge CRDTs lack.
- **The LCA cannot cover the second merge.** Once the death is processed, the deleted record leaves the visible horizon, and the ordering fact it justified must survive in the state: as a tombstone (RGA) or as one retained timestamp on the descendant (ours).

Status of the retention question. Machine-checked: remembering nothing breaks (8 refuted designs); remembering conditionally breaks (2 refuted rules, counterexamples above); always remembering works and is observationally equivalent to the tombstoned RGA, at one number per deleted character with living descendants — O(history) in the worst case, same as RGA. The one open question: whether the retained timestamps can be discarded once every replica has seen the deletion (causal stability), which would shrink the steady state to the live text plus in-flight activity.

Provenance. All rows reproduce from `whiteboard/litmus/contest_tree.py` (designs, both counterexamples as executable regressions, and the always-keep control's lockstep equivalence with the tombstoned RGA).